Tool Calling Intro

Modulo 3 · Lezione 7



Due domande a cui i documenti non rispondono

«Che giorno è oggi?»

> Oggi è il 27 aprile 2024.

sbagliato: oggi è un'altra data, e lo dice con sicurezza

«Quanto fa 849 diviso 1,22?»

il prezzo della LD-210 senza IVA

> 849 diviso 1,22 fa circa 696,72.

sbagliato: fa 695,90

Nessuno dei sei documenti Lumen contiene la data di oggi o il risultato di una divisione: il RAG delle ultime sei lezioni qui non aiuta.

> Il modello non sa cosa succede fuori dal suo testo e non fa i conti esatti. Stasera: come fargli **chiedere a noi** ciò che non sa fare.



Agenda

	Blocco	Dove	Tempo
	Apertura: le due domande	slide	5′
	Il giro del tool calling, tappa per tappa	slide	30′
	Codice spiegato: il giro con la calcolatrice (già nel file)	`working/tools.py`	15′
	Esercizio: il secondo tool (TODO)	`working/tools.py`	60′
	Recap e prossima lezione	slide	10′

Un solo file per tutta la serata: `working/tools.py`, dalla cartella `scripts/` (`python working/tools.py`). Stasera niente retrieval: `scaffolding/aikit/` è solo `llm_client.py`.

Setup come nelle lezioni scorse, prima del codice spiegato: venv attivo (`source .venv/bin/activate`), `pip install -r requirements.txt`, la chiave OpenAI in `scripts/.env` (i passi in `README.md`, sezione Setup).



Il modello chiede, non esegue



l'utente vede solo domanda e risposta: il resto passa tra il modello e il nostro codice

Con i tool il modello **non esegue codice** e non «ha accesso» a niente. Riceve, con la domanda, una lista di funzioni **nostre**: nome, a cosa serve, con quali argomenti.

il modello **non esegue niente**: restituisce una richiesta

eseguiamo noi, e gli diciamo com'è andata

Tutto il tool calling è questo giro, ripetuto finché il modello non ha più niente da chiedere.

Il giro in cinque tappe

Le cinque tappe, poi una per una sul contenuto vero della demo di stasera: «`Quanto fa 849 diviso 1,22?`». L'output del modello (`r.output`) è una lista: ogni elemento è un **item**, una richiesta o un messaggio.

`1 · definizione`
definiamo il tool: nome, descrizione, parametri

→

`2 · chiamata al modello`
nell'output c'è un item `function_call`

→

`3 · nostro codice`
eseguiamo la funzione

→

`4 · nostro codice`
rimandiamo un item `function_call_output`

→

`5 · chiamata al modello`
nell'output c'è il testo

se al posto del testo c'è un'altra `function_call`, si ripete da 3

viola: chiamate al modello, si pagano tutte e due

verde: il nostro programma, il modello non lo vede



Tappa 1: la definizione del tool

```
{
  "type": "function",
  "name": "calcolatrice",
  "description": "Esegue un'operazione aritmetica tra due
                  numeri e ritorna il risultato esatto.",
  "parameters": {
    "type": "object",
    "properties": {
      "a": {"type": "number"},
      "b": {"type": "number"},
      "operazione": {"type": "string", "enum": ["+", "-", "*", "/"]}
    },
    "required": ["a", "b", "operazione"]
  },
  "strict": false
}
```

`name`

come il modello lo chiama, e come noi lo riconosciamo

`description`

a cosa serve: è il testo con cui il modello decide **se** chiamarlo

`parameters`

gli argomenti, in JSON Schema: **come** chiamarlo

È un dizionario, e va nella lista `tools` di ogni chiamata. Il modello lo legge tutto. Nel file non lo scriviamo a mano: lo genera `definisci_tool()` da Pydantic, tra poche slide (qui senza i `title` e la `description` ripetuta che Pydantic aggiunge).



Tappa 2: la prima chiamata e l'item function_call

```python
r = client.responses.create(
    model="gpt-4.1-mini",
    instructions=ISTRUZIONI,
    input=[{"role": "user",
        "content": "Quanto fa 849 diviso 1,22?"}
    tools=TOOLS,
)
```

137 token in, 27 out. Nell'output **non c'è testo**: il modello ha deciso da solo che serviva il tool, e con quali argomenti. `ISTRUZIONI` è il system prompt («rispondi in italiano, in una frase») e vale a ogni chiamata del giro.

`r.output[0]`

```python
{
  "type": "function_call",
  "name": "calcolatrice",
  "arguments": '{"a":849,"b":1.22,"operazione":"/"}',
  "call_id": "call_3xSo7ra45k6vACkp6nwCCfMW"
}
```

`name`
quale tool

`arguments`
una **stringa** JSON: `json.loads()`

`call_id`
ci serve alla tappa 4



Tappe 3 e 4: eseguiamo e rimandiamo il risultato

3 · eseguiamo

```
argomenti = json.loads(chiamata.arguments)
risultato = calcolatrice(849, 1.22, "/")
# 695.9016393442623
```

Il modello non ha visto niente di tutto questo.

4 · rimandiamo

```
{
  "type": "function_call_output",
  "call_id": "call_3xSo7ra45k6vACkp6nwCCfMW",
  "output": "695.9016393442623"
}
```

Lo **stesso** `call_id` della richiesta; l'output come **stringa**.

LA REGOLA

Nell'input della seconda chiamata torna **tutto l'output della prima**, compresa la `function_call`, e poi il nostro output. Nel file la lista si chiama `storia`: `storia += r.output`, poi `storia.append(function_call_output)`.

Se rimandiamo solo l'output, l'API rifiuta: `400 · No tool call found for function call output.` Il modello deve rivedere cosa aveva chiesto.



Tappa 5: la seconda chiamata e la risposta

input della seconda chiamata

```
user · «Quanto fa 849 diviso 1,22?»
function_call · calcolatrice(849, 1.22, /)
function_call_output · "695.9016…"
```

stessi `tools`, stesso modello

→

`r.output[0]`

```
message
849 diviso 1,22 fa circa 695,90.
```

180 token in, 16 out. Niente `function_call` nell'output: **fine del giro**. Se ce ne fosse stata un'altra, si sarebbe ripetuto dalla tappa 3.

`chiamata 1` · 137 in, 27 out · esce la richiesta

`chiamata 2` · 180 in, 16 out · esce la risposta



Lo schema esce da Pydantic

A Modulo 2 · Lezione 13 un modello Pydantic descriveva la **risposta** che volevamo. Stasera descrive gli **argomenti** della funzione.

```python
class Calcolatrice(BaseModel):
    """Esegue un'operazione aritmetica tra due
    numeri e ritorna il risultato esatto."""
    a: float
    b: float
    operazione: Literal["+", "-", "*", "/"]
```

```python
def definisci_tool(schema, nome):
    return {
        "type": "function",
        "name": nome,
        "description": schema.__doc__,
        "parameters": schema.model_json_schema(),
        "strict": False,
    }
```

`Calcolatrice.model_json_schema()`

```json
{
  "type": "object",
  "properties": {
    "a": {"type": "number"},
    "b": {"type": "number"},
    "operazione": {"type": "string",
                   "enum": ["+", "-", "*", "/"]}
  },
  "required": ["a", "b", "operazione"]
}
```

La docstring è la descrizione. Il `Literal` diventa un `enum`: il modello non può inventare un'operazione. I tre campi finiscono in `required` (qui senza i `title` che Pydantic aggiunge a ogni campo).

`strict: False`: l'API non garantisce gli argomenti (con `True` sì, ma vuole uno schema più rigido). Chi vuole la garanzia: `Calcolatrice(**argomenti)`, e Pydantic valida.



Come il modello sceglie il tool

Per scegliere, il modello legge tutto quello che riceve: la domanda, le istruzioni e la lista dei tool. Di un tool conosce solo **nome, descrizione e parametri**: il codice della funzione non lo vede. Misurato su «Che giorno è oggi?», con il tool della data accanto alla calcolatrice, cinque giri per riga:

nome	descrizione	chiama il tool	risposta
`data_oggi`	`Ritorna la data di oggi…`	`5 su 5`	`la data di oggi`
`data_oggi`	`Uno strumento utile.`	`5 su 5`	`la data di oggi`
`strumento_2`	`Ritorna la data di oggi…`	`5 su 5`	`la data di oggi`
`strumento_2`	`Uno strumento utile.`	`0 su 5`	`«Oggi è il 27 aprile 2024.»`

`NOME O DESCRIZIONE`

Basta che uno dei due dica a cosa serve il tool. Quando nessuno dei due lo dice, il modello non lo chiama e torna a inventare.

Anche i nomi dei parametri contano (`a`, `b`, `operazione` dicono qualcosa). Questa prova la farete voi nell'esercizio, sulla domanda «Che giorno è oggi?».



Due tool nella stessa risposta

«Che giorno è oggi, e quanto fa 849 per 7?»: servono tutti e due, e il modello può chiederli **insieme**.

chiamata 1 · r.output

`function_call · data_oggi({}) · call_id A`

`function_call · calcolatrice(849, 7, *) · call_id B`

il nostro codice, un ciclo for sulle chiamate

`function_call_output · "‹la data di oggi›" · call_id A`

`function_call_output · "5943" · call_id B`

chiamata 2 · r.output

message · Oggi è ‹la data di oggi›, e 849 per 7 fa 5943.

Due richieste, due risultati, ognuno col suo `call_id`, una sola seconda chiamata. Per questo nel codice le `function_call` si scorrono con un `for`, non si prende la prima.



Il costo del giro

Ogni giro sono almeno **due chiamate**, e si pagano entrambe: la seconda porta dentro tutto l'input della prima più gli item nuovi.

domanda	chiamate	token	costo	risposta
«849 diviso 1,22», senza tool	1	39 in · 15 out	$0,00004	**sbagliata** (696,72)
«849 diviso 1,22», la calcolatrice in `TOOLS`	2	137 + 180 in · 27 + 16 out	$0,00020	giusta (695,90)
«849 diviso 1,22», due tool in `TOOLS` (dopo l'esercizio)	2	162 + 205 in · 27 + 16 out	$0,00022	giusta (695,90)
«Che giorno è oggi, e 849 per 7?», due tool	2	165 + 222 in · 53 + 23 out	$0,00028	giusta
«Cos'è una scrivania regolabile?», due tool in `TOOLS`	1	163 in · 40 out	$0,00013	giusta dalla sua conoscenza, nessun tool

Le definizioni dei tool stanno nell'input di **ogni chiamata**, anche quando non servono: il secondo tool sono 25 token in più a chiamata (137 → 162).

L'ultima riga non rifiuta: stasera non c'è retrieval, il modello risponde da quello che sa (a Modulo 3 · Lezione 9 sarà il sistema a decidere se cercare o rispondere). Con Anthropic il giro è lo stesso: cambiano i nomi degli item (`tool_use`, `tool_result`).



La struttura di working/tools.py

`1 Calcolatrice` + calcolatrice()

`TODO DataOggi` + data_oggi()

`2 definisci_tool()` + TOOLS

`3 esegui(nome, argomenti)`

`4 chiedi(domanda)` il giro

`stampa_chiamata, stampa_esecuzione, stampa_costo`

`main` le quattro DOMANDE, o SOLO

lo schema degli argomenti e la funzione vera → **tappa 1**

il secondo tool: lo scrivete voi, nell'esercizio

dallo schema al dizionario che l'API vuole; la lista che il modello vede → **tappa 1**

un `if` sul nome: esegue il tool chiesto → **tappa 3**

chiama, esegue, rimanda, ripete → **tappe 2, 4, 5**

ogni passo del giro a schermo, con token e costo

solo calcolo · solo data · entrambe · nessuna

Tutto già scritto tranne il TODO. Ora lo apriamo.



Codice spiegato: il giro con la calcolatrice

CODICE SPIEGATO — il docente spiega, voi seguite sul vostro file

`working/tools.py` · tutto già scritto tranne il TODO · 15 minuti

si rientra dopo il lancio, con l'output a schermo

COSA C'È NEL FILE

`MODELLO` gpt-4.1-mini · `ISTRUZIONI` · le quattro `DOMANDE` e `SOLO` · la calcolatrice · `definisci_tool()` e `TOOLS` · `esegui()` · `chiedi()` · gli helper di stampa · il main

`4 · chiedi(domanda)`, le tappe nel codice

`responses.create(…, input=storia, tools=TOOLS)` → la chiamata (tappe 2 e 5)

`storia += r.output` → la regola della tappa 4

nessuna `function_call` nell'output → si ritorna il testo; altrimenti, per ognuna: `esegui()` e `function_call_output` (tappe 3 e 4)

Poi lo lancia il docente: `python working/tools.py`, le quattro domande, ogni passo del giro a schermo. Chi ha già `.env` pronto lo lancia anche lui.



Cosa si è visto: il giro a schermo

```
D: Quanto fa 849 diviso 1,22?
   chiamata 1  [137 token in, 27 out]  output:
      function_call calcolatrice({"a":849,"b":1.22,"operazione":"/"})
      eseguo calcolatrice({'a': 849, 'b': 1.22, 'operazione': '/'}) = 695.9016…
   chiamata 2  [180 token in, 16 out]  output: message
   2 chiamate · $0.00020
→ 849 diviso 1,22 fa circa 695,90.

D: Che giorno è oggi?
   chiamata 1  [131 token in, 13 out]  output: message
   1 chiamata · $0.00007
→ Oggi è il 27 aprile 2024.

D: Che giorno è oggi, e quanto fa 849 per 7?
   chiamata 1 · function_call calcolatrice(…) · chiamata 2 · message
→ Oggi è il 27 aprile 2024, e 849 per 7 fa 5943.

D: Cos'è una scrivania regolabile in altezza?
   chiamata 1 · message · → risponde da quello che sa, nessun tool
```

IL GIRO A SCHERMO

Prima chiamata: la richiesta. Eseguiamo. Seconda chiamata: la risposta, giusta. Due chiamate, $0,0002: i token della teoria, perché in T00LS c'è solo la calcolatrice.

IL TOOL CHE MANCA

Sulla data il modello non ha niente da chiedere, e inventa una data del 2024: anche nella terza domanda, dove il conto è giusto. La quarta non chiama niente. Il tool lo scrivete voi.



Esercizio: il secondo tool

SCRIVETE VOI — da soli, con affiancamento

`working/tools.py` · il TODO in tre punti · 60 minuti (20' il TODO · 10' le quattro domande · 15' la prova · il resto l'extra) · consegna in `CONSEGNA.md`

si rientra per il recap

1 · lo schema e la funzione

`class DataOggi(BaseModel)`, senza campi: la docstring è la descrizione. `data_oggi()` ritorna `date.today().isoformat()`.

2 · la voce in TOOLS

`definisci_tool(DataOggi, "data_oggi")`

3 · il ramo in esegui()

`if nome == "data_oggi": return data_oggi()`

poi la prova

Rilancia: quali tool chiama il modello per le quattro domande? Poi nome `"strumento_2"` (in `TOOLS` e nell'`if`, devono combaciare) e docstring `"Uno strumento utile."`: rilancia e guarda la **seconda** domanda (sulla terza il modello, avendo già deciso di usare la calcolatrice, chiama anche il tool generico). Poi cambiane uno solo dei due. Errori tipici e «se finisci prima»: `CONSEGNA.md`.

FATTO QUANDO

1 · Con nome e descrizione buoni le quattro domande usano i tool giusti (calcolatrice · data · entrambi · nessuno) e la seconda dà **la data di oggi**.
2 · Con nome e descrizione generici la seconda torna a una data del 2024, e l'hai visto sul tuo schermo.



Cosa si è visto: il secondo tool e la prova

```
D: Che giorno è oggi?
   chiamata 1  [156 token in, 12 out]  output:
      function_call data_oggi({})
      eseguo data_oggi({}) = ‹la data di oggi›
   chiamata 2  [183 token in, 13 out]  output: message
   2 chiamate · $0.00018
→ Oggi è ‹la data di oggi›.

D: Che giorno è oggi, e quanto fa 849 per 7?
   chiamata 1  [165 token in, 53 out]  output:
      function_call data_oggi({}) · function_call calcolatrice(…)
      eseguo data_oggi({}) = ‹la data di oggi›
      eseguo calcolatrice({'a': 849, 'b': 7, 'operazione': '*'}) = 5943
   chiamata 2  [222 token in, 23 out]  output: message
→ Oggi è ‹la data di oggi›, e 849 per 7 fa 5943.
```

la prova, 5 giri per riga

nome e descrizione generici	0 su 5	data inventata
uno solo dei due generico	5 su 5	la data di oggi

È la tabella della slide sulla scelta del tool, rifatta sul vostro schermo con un lancio per configurazione (i cinque giri li abbiamo fatti noi): la prova si giudica sulla seconda domanda, quella sola sulla data.



Recap: il giro

1 · definizione
nome, descrizione, parametri da Pydantic

```
definisci_tool(schema, nome)
```

2 · chiamata
esce una `function_call`: nome, argomenti, call_id

```
responses.create(…, tools=TOOLS)
```

3 · eseguiamo
noi, nel nostro programma

```
esegui(name, json.loads(arguments))
```

4 · rimandiamo
un `function_call_output` con lo stesso call_id, dopo tutto l'output

```
storia += r.output; storia.append(…)
```

5 · chiamata
esce il testo, o un'altra richiesta

```
return r.output_text
```

il modello **chiede**, non esegue

almeno **due chiamate**, pagate entrambe

di un tool conosce solo **nome, descrizione, parametri**



Cosa la demo lascia fuori

La calcolatrice non può fallire, non tocca dati, non ha bisogno di sicurezza. I tool veri sì.

ERRORI

Un tool che alza un'eccezione ferma il giro. Un tool che risponde «cliente non trovato» lascia decidere al modello.

DATI VERI

Un database, una ricerca web. E un tool che restituisce 4.000 token di JSON costa e affoga il contesto.

SICUREZZA

Un tool che esegue SQL o codice scritto dal modello è un rischio: come si fa in sola lettura.

> Tutto questo è la prossima lezione, **nello stesso file**.

il giro gestito da un SDK → più avanti nel corso

agenti che lavorano da soli su più passi, MCP → Modulo 4



La prossima lezione

stasera
il giro, scritto a mano, su due tool giocattolo

→

Modulo 3 · Lezione 8 — Tool Implementation
stesso file, `working/tools.py`: i tool toccano un database vero e una ricerca web; descrizione, errori e sicurezza smettono di essere dettagli

Poi il lab (Modulo 3 · Lezione 9): il RAG del modulo diventa uno dei tool, e il sistema decide da solo se cercare nei documenti, chiamare un tool o rispondere direttamente.



Grazie

prossima lezione: Modulo 3 · Lezione 8 — Tool Implementation

